Solutions Manual for Extracted Exercises

Extracted from the uploaded AI textbook source

Contents

1 1.1 What Does It Mean to Learn? 2

2 1.2 A Mathematical Formulation of Learning 4

3 1.3 Ideal Prediction, Bayes Rules, and Learnable Models 5

4 1.4 Generalization: Why Fitting the Data Is Not Enough 7

5 1.5 Optimization as the Engine of Learning 9

6 1.6 Main Learning Paradigms 11

7 1.7 From Classical Prediction to Representation Learning 12

8 2.1 From Classical Prediction to Learned Representations 14

9 2.2 Neural Networks as Compositional Function Models 15

10 2.3 Expressivity, Hierarchy, and the Role of Depth 17

11 2.4 Computation Graphs, Chain Rule, and Backpropagation 19

12 2.5 Why Deep Networks Are Hard to Train 21

13 2.6 Optimization in Practice: SGD, Initialization, and Signal Propagation 22

14 3.3 Multiscale Representation and the Fourier Viewpoint 23

15 3.4 Sequential Data and Recurrent Computation 25

16 3.5 Gating and the Control of Memory 27

17 3.6 From Recurrence to Attention 28

18 3.7 Transformers as General-Purpose Sequence Architectures 30

19 3.8 Scaling Transformer Architectures and Multimodal Extensions 32

20 3.9 Worked Examples and Comparative Case Studies 33

21 3.10 What Is Understood and What Remains Open **35**

22 4.1 From Task-Specific Prediction to Reusable Representation **37**

23 4.5 Representation Learning as a Unifying Principle **38**

24 4.6 State Representation in Sequential Decision-Making **39**

25 4.7 Deep Reinforcement Learning as Learned Representation Plus Control **41**

26 5.4 Adversarial Generative Modeling **42**

1 1.1 What Does It Mean to Learn?

Exercise 1

Exercise. For each of the following scenarios, say whether the central goal is primarily one of *prediction*, *decision*, or *representation*. Briefly justify your answer.

1. Predicting whether an image contains a tumor.
2. Choosing a dosage policy for a patient over time.
3. Learning a low-dimensional embedding of customer behavior.
4. Predicting tomorrow's electricity demand.
5. Selecting moves in a board game to maximize the chance of winning.

Solution.

1. Tumor detection: primarily *prediction*, because the model estimates whether a tumor is present.
2. Dosage policy over time: primarily *decision*, because one must choose actions that affect outcomes.
3. Low-dimensional embedding of customer behavior: primarily *representation*, because the goal is a useful encoding.
4. Electricity-demand forecasting: *prediction*.
5. Choosing moves in a board game: *decision*, because the objective is action selection to maximize long-run reward.

Exercise 2

Exercise. Write down plausible triples (E, T, P) for each of the following:

1. a regression problem,
2. a generative modeling problem,
3. a reinforcement learning problem.

Be explicit about what counts as experience, what the task is, and how performance is measured.

Solution. One possible set of triples is:

1. Regression: E = labeled input-output pairs (x_i, y_i), T = predict a real-valued y from x, P = mean squared error on fresh data.

2. Generative modeling: E = unlabeled samples $x_i \sim P_X$, T = model or sample from the data distribution, P = log-likelihood, held-out likelihood, or sample quality.

3. Reinforcement learning: E = trajectories (s_t, a_t, r_t) collected by interaction, T = choose actions to maximize return, P = expected cumulative reward.

Exercise 3

Exercise. Construct a predictor that attains zero error on a finite dataset but is clearly unreasonable on new points. Explain why this illustrates the difference between memorization and learning.

Solution. For a finite training set $\{(x_i, y_i)\}_{i=1}^n$, define

$$f(x) = \begin{cases} y_i, & x = x_i \text{ for some training point,} \\ 0, & \text{otherwise.} \end{cases}$$

This achieves zero training error but is unreasonable off the sample because it contains no smoothness, invariance, or structural assumption. It memorizes rather than learns a rule that transfers.

Exercise 4

Exercise. A hospital uses a model to assign a risk score to patients and then uses that score to decide which patients receive immediate follow-up. Identify which part of the pipeline is prediction and which part is decision. Give one reason why optimizing accuracy alone may be insufficient in this setting.

Solution. The risk score is the *prediction*; the triage rule that decides who gets immediate follow-up is the *decision*. Accuracy alone may be insufficient because false negatives and false positives have very different costs, and fairness or resource constraints may matter.

Exercise 5

Exercise. Give an example in which the same dataset could support two different tasks with two different performance criteria. Explain why the learning problem changes when the task or criterion changes, even if the data do not.

Solution. Using the same medical dataset, one task could be *predict disease status* with performance measured by classification error or AUC; another could be *rank patients for screening* with performance measured by recall at a fixed budget. The dataset is unchanged, but the target object and success criterion differ, so the learning problem itself changes.

2 1.2 A Mathematical Formulation of Learning

Exercise 1

Exercise. Give one example of a learning problem in which the output space $\mathcal{Y}$ is discrete and one in which it is continuous. Suggest a natural loss function in each case.

Solution. A discrete-output example is spam detection with $\mathcal{Y} = \{0, 1\}$; a natural loss is cross-entropy or zero–one loss. A continuous-output example is house-price prediction with $\mathcal{Y} \subseteq \mathbb{R}$; a natural loss is squared loss or absolute error.

Exercise 2

Exercise. Let $\ell(\hat{y}, y) = (\hat{y} - y)^2$. Compute the empirical risk of the constant predictor $f_c(x) = c$ on a dataset $\{(x_i, y_i)\}_{i=1}^n$ and verify directly that the minimizing value of c is the sample mean.

Solution. The empirical risk is

$$\widehat{R}_n(c) = \frac{1}{n} \sum_{i=1}^{n} (c - y_i)^2.$$

Differentiating gives

$$\widehat{R}_n'(c) = \frac{2}{n} \sum_{i=1}^{n} (c - y_i) = 2\Big(c - \frac{1}{n} \sum_{i=1}^{n} y_i\Big).$$

Setting this to zero yields

$$c^* = \bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i,$$

and the second derivative is $2 > 0$, so the sample mean minimizes the empirical risk.

Exercise 3

Exercise. Explain in words why the proposition on unbiasedness for fixed f does not imply that the training error of the empirical risk minimizer is an unbiased estimate of its test error.

Solution. The unbiasedness proposition assumes the predictor f is fixed independently of the sampled data. The empirical risk minimizer $\hat{f}_n$ is chosen *using the same sample* on which its training error is measured, so selection bias appears: the model is adapted to that sample and its training error is therefore optimistically biased.

Exercise 4

Exercise. Describe how inductive bias enters through the choice of hypothesis class $\mathcal{H}$ even before any data are observed.

Solution. Inductive bias enters before data through the choice of $\mathcal{H}$. If $\mathcal{H}$ contains only linear functions, the learner assumes approximate linearity; if it contains smooth functions, convolutional networks, or trees, it assumes different structure. Data select within the class, but the class itself already constrains what can be learned.

Exercise 5

Exercise. Consider a classification problem with zero–one loss. Write down the expected risk and empirical risk of a predictor $f : \mathcal{X} \to \{0, 1\}$. What does minimizing each quantity mean in plain language?

Solution. With zero–one loss,

$$R(f) = \mathbb{E}[\mathbf{1}\{f(X) \neq Y\}], \qquad \widehat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\{f(x_i) \neq y_i\}.$$

Minimizing expected risk means minimizing misclassification probability under the true distribution. Minimizing empirical risk means minimizing the fraction of mistakes on the observed sample.

Exercise 6

Exercise. In the approximation–estimation–optimization decomposition, describe a concrete situation in which each of the three terms could be large.

Solution. Approximation can be large if the true relationship is highly nonlinear but $\mathcal{H}$ is linear. Estimation can be large if the class is flexible but the dataset is tiny, so the selected model is unstable. Optimization can be large if the training procedure stops early, uses a bad learning rate, or gets stuck far from a good minimizer.

3 1.3 Ideal Prediction, Bayes Rules, and Learnable Models

Exercise 1

Exercise. Let $\ell(\hat{y}, y) = (\hat{y} - y)^2$. Starting from

$$\mathbb{E}[(a - Y)^2 \mid X = x],$$

show directly that the minimizing value of a is $\mathbb{E}[Y \mid X = x]$.

Solution. Expand around the conditional mean $m(x) = \mathbb{E}[Y \mid X = x]$:

$$\mathbb{E}[(a - Y)^2 \mid X = x] = \mathbb{E}[(a - m + m - Y)^2 \mid X = x].$$

This equals

$$(a - m)^2 + 2(a - m)\mathbb{E}[m - Y \mid X = x] + \mathbb{E}[(m - Y)^2 \mid X = x].$$

The middle term is zero, so

$$\mathbb{E}[(a - Y)^2 \mid X = x] = (a - m)^2 + \mathrm{Var}(Y \mid X = x),$$

which is minimized at $a = m(x) = \mathbb{E}[Y \mid X = x]$.

Exercise 2

Exercise. Suppose $\mathbb{P}(Y = 1 \mid X = x) = p(x)$ in a binary classification problem. Under zero–one loss, write down the Bayes classifier explicitly and explain why the threshold is 1/2.

Solution. Under zero–one loss, predict the more likely class:

$$f^*(x) = \begin{cases} 1, & p(x) \geq \frac{1}{2}, \\ 0, & p(x) < \frac{1}{2}. \end{cases}$$

The threshold is 1/2 because the conditional error of predicting 1 is $1-p(x)$, while that of predicting 0 is $p(x)$; choose the smaller of the two.

Exercise 3

Exercise. Give an example of two different loss functions that would lead to two different notions of ideal prediction for the same conditional distribution of $Y \mid X = x$.

Solution. For the same $Y \mid X = x$, squared loss leads to the conditional mean, while absolute loss leads to a conditional median. If the conditional distribution is skewed, these are different ideal predictions.

Exercise 4

Exercise. Explain in words why Bayes-optimal prediction is a useful theoretical benchmark even though the learner does not know the true distribution.

Solution. The Bayes predictor is useful because it defines the best possible performance under the assumed distribution and loss. Even though the learner does not know the distribution, it gives a target for comparison: approximation, estimation, and optimization errors can all be understood relative to this ideal benchmark.

Exercise 5

Exercise. Suppose a regression problem satisfies

$$Y = g(X) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0.$$

Show that under squared loss the Bayes-optimal predictor is $g(X)$.

Solution. Since $Y = g(X) + \varepsilon$ with $\mathbb{E}[\varepsilon \mid X] = 0$,

$$\mathbb{E}[Y \mid X] = g(X) + \mathbb{E}[\varepsilon \mid X] = g(X).$$

Under squared loss the Bayes predictor is the conditional mean, so the Bayes-optimal predictor is $g(X)$.

Exercise 6

Exercise. For a three-class problem, suppose that at some input x,

$$\mathbb{P}(Y = 1 \mid X = x) = 0.4, \qquad \mathbb{P}(Y = 2 \mid X = x) = 0.35, \qquad \mathbb{P}(Y = 3 \mid X = x) = 0.25.$$

What is the Bayes-optimal prediction under zero–one loss, and what is the corresponding conditional Bayes error?

Solution. The Bayes-optimal class is 1, since 0.4 is the largest posterior probability. The conditional Bayes error is

$$1 - \max\{0.4, 0.35, 0.25\} = 1 - 0.4 = 0.6.$$

So even the optimal classifier makes an error with conditional probability 0.6 at that x.

4 1.4 Generalization: Why Fitting the Data Is Not Enough

Exercise 1

Exercise. Define in your own words the difference between training error, test error, and the generalization gap. Why can a model have small training error but large test error?

Solution. Training error is performance on the training sample; test error is performance on new unseen data from the same distribution; the generalization gap is their difference. A model can have small training error but large test error when it matches idiosyncrasies or noise in the sample rather than a stable pattern in the population.

Exercise 2

Exercise. Construct a simple example of a predictor that memorizes a finite training set. Explain why zero training error does not certify learning in the predictive sense.

Solution. A memorizing predictor can store the lookup table of all training inputs and labels and default arbitrarily elsewhere. It achieves zero training error, but because it has no principled behavior off the sample, predictive learning is not certified.

Exercise 3

Exercise. Suppose two model classes are available for the same task: one very simple and one highly flexible. Describe a situation in which the simple model underfits and a situation in which the flexible model overfits.

Solution. A simple linear model underfits if the truth is strongly nonlinear, such as concentric circles. A very flexible model overfits when the sample is small and noisy, for example a high-degree polynomial interpolating noisy scalar data.

Exercise 4

Exercise. Cross-validation is often used to choose hyperparameters such as model complexity or regularization strength. Explain conceptually why evaluating all candidate models only on the training set is unreliable, and how cross-validation attempts to correct this.

Solution. Evaluating all candidates only on training data is unreliable because more flexible models can win by fitting noise. Cross-validation repeatedly withholds part of the data, trains on the rest, and evaluates on held-out folds, thereby approximating out-of-sample performance for model selection.

Exercise 5

Exercise. In the decomposition

$$R(\hat{f}_n) - R(f^\star) = \text{approximation} + \text{estimation} + \text{optimization},$$

describe one concrete reason each term might be large in practice.

Solution. Approximation may be large because the class is misspecified. Estimation may be large because data are limited relative to model flexibility. Optimization may be large because SGD, initialization, or conditioning prevent the algorithm from finding a near-best point in the class.

Exercise 6

Exercise. Why is it misleading to define overfitting simply as "using too many parameters"? Give at least two other factors that influence whether a model generalizes well.

Solution. Parameter count alone is not decisive. Generalization also depends on data size, noise level, regularization, optimization bias, architecture, augmentation, and whether the parameters encode useful inductive bias rather than arbitrary flexibility.

5 1.5 Optimization as the Engine of Learning

Exercise 1

Exercise. Let

$$J(\theta) = \frac{1}{2}(\theta - 5)^2.$$

Compute $J'(\theta)$ and write the gradient descent update with learning rate η. Starting from $\theta_0 = 1$, compute θ_1 and θ_2 when $\eta = \frac{1}{2}$.

Solution. $J'(\theta) = \theta - 5$, so the gradient descent update is

$$\theta_{k+1} = \theta_k - \eta(\theta_k - 5).$$

With $\eta = \frac{1}{2}$ and $\theta_0 = 1$,

$$\theta_1 = 1 - \tfrac{1}{2}(1 - 5) = 3, \qquad \theta_2 = 3 - \tfrac{1}{2}(3 - 5) = 4.$$

Exercise 2

Exercise. Consider the linear model

$$f_\theta(x) = \theta x$$

with squared loss on a single example (x, y). Show that

$$\ell(\theta; (x, y)) = (\theta x - y)^2$$

has derivative

$$\frac{d}{d\theta}\ell(\theta; (x, y)) = 2x(\theta x - y).$$

Solution. Differentiate by the chain rule:

$$\ell(\theta; (x, y)) = (\theta x - y)^2.$$

Let $u(\theta) = \theta x - y$. Then $\ell = u^2$ and $u'(\theta) = x$, so

$$\frac{d\ell}{d\theta} = 2u\,u' = 2(\theta x - y)x = 2x(\theta x - y).$$

Exercise 3

Exercise. Explain in words why the negative gradient is a sensible update direction for a differentiable objective.

Solution. For a differentiable objective, the gradient is the direction of steepest local increase, so the negative gradient is the direction of steepest local decrease to first order. A small step along $-\nabla J(\theta)$ therefore reduces the objective most efficiently among infinitesimal moves.

Exercise 4

Exercise. What is the practical difference between full-batch gradient descent and stochastic gradient descent? What is the role of a minibatch?

Solution. Full-batch gradient descent computes the exact gradient over the whole training set each step; SGD uses one example or a small random batch, giving a noisy but cheaper gradient estimate. A minibatch balances variance and cost: it reduces noise relative to single-example SGD while remaining much cheaper than full-batch updates.

Exercise 5

Exercise. Give an example of a situation in which a model achieves very low training loss but still generalizes poorly. Which concept from the previous section does this illustrate?

Solution. For example, a large neural network can interpolate noisy labels and achieve near-zero training loss yet perform poorly on a clean test set. This illustrates overfitting and a large generalization gap.

Exercise 6

Exercise. Why does convexity make optimization easier to reason about, at least conceptually, than a general nonconvex objective?

Solution. In a convex problem every local minimum is global, so the landscape is conceptually easier: one can reason about convergence and solution quality without worrying that optimization found a bad basin. In a nonconvex problem there may be many critical points and optimization behavior is harder to characterize globally.

6 1.6 Main Learning Paradigms

Exercise 1

Exercise. For each of the following, identify the most natural learning paradigm and briefly justify your answer:

1. predicting house prices from past sales data,
2. grouping customers into behavioral segments without labels,
3. predicting a masked word in a sentence,
4. learning to control a robot arm from reward signals,
5. reconstructing an image from a compressed latent code.

Solution.

1. House prices from past sales: supervised learning.
2. Customer segmentation without labels: unsupervised learning.
3. Predicting a masked word: self-supervised learning.
4. Control from reward signals: reinforcement learning.
5. Reconstructing an image from a compressed latent code: usually unsupervised or self-supervised representation learning, as in an autoencoder.

Exercise 2

Exercise. Write a toy self-supervised objective for one of the following settings:

1. a sentence with one hidden token,
2. an image with one missing patch,
3. a time series with a short future segment to be predicted.

Be explicit about what serves as the input and what serves as the target.

Solution. One text example is masked-token prediction. Input: a sentence such as "The cat sat on the [MASK]." Target: the hidden token "mat." A toy objective is cross-entropy between the model's predicted distribution for the masked position and the true token.

Exercise 3

Exercise. Explain why reinforcement learning is not simply supervised learning on a dataset of trajectories. In your answer, discuss at least two of the following: delayed reward, exploration, action-dependent data, and long-term consequences.

Solution. RL is not just supervised learning on trajectories because the data depend on the policy's actions. Rewards may be delayed, so the correct target for an action is not an immediate label but a long-run return. Exploration is required to gather informative data, and actions change future states and hence future data distributions.

Exercise 4

Exercise. Give one example of an unsupervised objective and one example of a self-supervised objective. Why is it helpful to distinguish these two regimes instead of merging them into a single category?

Solution. An unsupervised objective is k-means clustering or density estimation on unlabeled data. A self-supervised objective is predicting a masked token from surrounding words. The distinction matters because self-supervision manufactures predictive targets from structure in the input, while generic unsupervised learning need not do so.

Exercise 5

Exercise. A modern model is pretrained by predicting missing tokens on a large text corpus and then fine-tuned on labeled sentiment examples. Which paradigms appear in this pipeline, and what role does each one play?

Solution. The pretraining stage is self-supervised learning: the model learns broad language representations by predicting missing tokens. The fine-tuning stage is supervised learning: labeled sentiment examples adapt those representations to a specific downstream task.

7 1.7 From Classical Prediction to Representation Learning

Exercise 1

Exercise. Give an example of a learning problem in which a simple predictor becomes much more effective after a suitable feature transformation. Describe both the raw input space and the transformed feature space.

Solution. A standard example is XOR. In raw coordinates (x_1, x_2), a linear classifier fails. After the feature map $\phi(x) = (x_1, x_2, x_1x_2)$ or a suitable hidden representation, a simple linear separator in feature space can separate the classes.

Exercise 2

Exercise. Consider the feature map

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2).$$

Explain why a linear classifier in the feature space can represent decision boundaries that are not linear in the original input space.

Solution. A linear classifier in feature space has decision rule

$$a_1 x_1 + a_2 x_2 + a_3(x_1^2 + x_2^2) + b = 0.$$

In the original coordinates this is generally quadratic, for example a circle or annulus-type boundary, so it is not restricted to a straight line.

Exercise 3

Exercise. In your own words, explain the difference between

$$f(x) = g(\phi(x))$$

with fixed ϕ and

$$f_\theta(x) = g_{\theta_2}(\phi_{\theta_1}(x))$$

with learned ϕ_{θ_1}.

Solution. With fixed ϕ, the representation is chosen in advance and only the downstream predictor g is learned. With learned ϕ_{θ_1}, the representation itself adapts to the task, so the system can reshape the input geometry before the final predictor acts.

Exercise 4

Exercise. Why is it reasonable to say that representation learning changes the geometry of the problem? Give one example of a property that may become simpler in representation space than in raw space.

Solution. A representation map can make difficult structure simpler: classes that are tangled in pixel space may become linearly separable in feature space; nuisance variation such as translation or lighting may be compressed; or a low-dimensional manifold may be flattened into coordinates useful for prediction.

Exercise 5

Exercise. A neural network ends with a linear output layer. Why is it still not "just a linear model" in general?

Solution. Because the final linear layer acts on *learned nonlinear features*. The overall map is linear only if all preceding layers are affine. Once hidden layers contain nonlinear activations, the composition with the final linear map is generally nonlinear in the original input.

8 2.1 From Classical Prediction to Learned Representations

Exercise 1

Exercise. A classifier has score $s(x) = w^\top x + b$. Explain in words how the same score can underlie a perceptron, logistic regression, and a linear SVM while still leading to different learning behavior.

Solution. The score $s(x) = w^\top x + b$ defines the same separating hyperplane in all three models, but the learning rule differs. The perceptron uses mistake-driven updates, logistic regression minimizes log-loss and yields probabilistic scores, and the linear SVM minimizes a margin-based objective with hinge-style penalties and explicit norm control.

Exercise 2

Exercise. Suppose a separator correctly classifies all training points. Why does that not settle the learning problem from a margin viewpoint?

Solution. No. From a margin viewpoint many separators may classify all points correctly, but some place the boundary dangerously close to data. Margin asks not just for correctness but for *robust* separation, preferring a separator that leaves a wider buffer around the classes.

Exercise 3

Exercise. Show directly that replacing (w, b) by (cw, cb) for $c > 0$ leaves the decision boundary unchanged. Then explain why this makes the geometric margin more meaningful than the functional margin.

Solution. For any $c > 0$,

$$(cw)^\top x + cb = c(w^\top x + b),$$

so the sign, and hence the decision boundary $\{x : w^\top x + b = 0\}$, is unchanged. The functional margin changes under this scaling, so it is not geometrically intrinsic. The geometric margin divides out the irrelevant scale of (w, b) and therefore reflects actual distance to the separating hyperplane.

Exercise 4

Exercise. In the soft-margin objective

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i,$$

explain informally what happens when C is chosen extremely large and when it is chosen very small.

Solution. If C is extremely large, the optimizer heavily penalizes slack and tries hard to fit all points, even at the cost of a smaller margin and possibly poorer robustness. If C is very small, margin size is emphasized and violations are tolerated more freely, producing a softer boundary that may ignore some training errors.

Exercise 5

Exercise. Consider the XOR pattern in $\mathbb{R}^2$. Why can a linear classifier fail in the original coordinates while a kernel method or a learned hidden representation can succeed?

Solution. XOR is not linearly separable in the original plane, so a linear classifier cannot separate the labels. A kernel method effectively lifts the points to a higher-dimensional feature space where a linear separator may exist, and a neural network can learn a hidden representation that performs a similar geometric untangling.

Exercise 6

Exercise. Using the comparison table, explain in your own words why SVM is best understood as the culmination of the classical fixed-feature pipeline rather than as a first deep-learning model.

Solution. SVM stays within the fixed-feature classical pipeline: one chooses features (or a kernel) and then solves a convex margin-based optimization problem on top of them. Deep learning, by contrast, learns the representation itself through layered nonlinear composition, so SVM is a culmination of classical geometric learning rather than a deep representation learner.

9 2.2 Neural Networks as Compositional Function Models

Exercise 1

Exercise. Run the perceptron algorithm by hand on the following dataset in $\mathbb{R}^2$:

$$x_1 = (1,1),\ y_1 = 1, \qquad x_2 = (-1,-1),\ y_2 = -1.$$

Start from $w_0 = 0$, $b_0 = 0$, process the examples cyclically, and compute the first few updates until the dataset is classified correctly.

Solution. Using the perceptron update $w \leftarrow w + y_i x_i$, $b \leftarrow b + y_i$, start from $(w, b) = (0, 0)$. Treat the first point as a mistake at zero score. After (x_1, y_1),

$$w = (1,1), \qquad b = 1.$$

Now the second point has score $(1,1) \cdot (-1,-1) + 1 = -1$, so it is correctly classified, and the first point has score $(1,1) \cdot (1,1) + 1 = 3$, also correct. So one update suffices.

Exercise 2

Exercise. Show that the bias term can be absorbed into the weight vector by augmenting the input with a constant 1. More precisely, prove that

$$w^\top x + b = \tilde{w}^\top \tilde{x}$$

for suitable augmented vectors $\tilde{w}$ and $\tilde{x}$.

Solution. Take

$$\tilde{x} = \begin{bmatrix} x \\ 1 \end{bmatrix}, \qquad \tilde{w} = \begin{bmatrix} w \\ b \end{bmatrix}.$$

Then

$$\tilde{w}^\top \tilde{x} = w^\top x + b \cdot 1 = w^\top x + b.$$

So the bias can be absorbed by augmenting the input with a constant coordinate.

Exercise 3

Exercise. Explain why the XOR labeling of the four points

$$(0,0),\ (0,1),\ (1,0),\ (1,1)$$

is not linearly separable.

Solution. If XOR were linearly separable, the positive points $(0,1)$ and $(1,0)$ would lie on one side of a line and the negative points $(0,0)$ and $(1,1)$ on the other. But the convex hull of the positives intersects the convex hull of the negatives at $(1/2, 1/2)$, so no hyperplane can separate them.

Exercise 4

Exercise. Prove directly that the composition of three affine maps is affine.

Solution. Let $f(x) = A_1x + b_1$, $g(x) = A_2x + b_2$, and $h(x) = A_3x + b_3$. Then

$$h(g(f(x))) = A_3(A_2(A_1x + b_1) + b_2) + b_3 = (A_3A_2A_1)x + (A_3A_2b_1 + A_3b_2 + b_3),$$

which is affine.

Exercise 5

Exercise. Why does the proposition on affine composition show that nonlinear activations are essential in deep learning?

Solution. Because if all layers were affine, their composition would still be affine, no matter how many layers were stacked. Nonlinear activations are therefore what allow deep networks to represent genuinely nonlinear decision boundaries and functions.

Exercise 6

Exercise. Consider a perceptron $f(x) = \text{sign}(w^\top x + b)$. What geometric object is defined by the equation

$$w^\top x + b = 0?$$

How does the sign of $w^\top x + b$ determine the prediction?

Solution. The equation $w^\top x + b = 0$ defines a hyperplane (a line in $\mathbb{R}^2$). Points with positive score lie on one side and are assigned one class; points with negative score lie on the other side and are assigned the opposite class.

10 2.3 Expressivity, Hierarchy, and the Role of Depth

Exercise 1

Exercise. In your own words, explain the difference between saying that a model class is expressive and saying that it is easy to train.

Solution. Expressive means the hypothesis class can represent many functions, possibly including the target. Easy to train means optimization can actually find a good parameter setting efficiently. A class may be expressive yet hard to optimize, so representability and trainability are different issues.

Exercise 2

Exercise. State the universal approximation theorem informally. Then list two things that the theorem does *not* imply.

Solution. Informally, the universal approximation theorem says that a sufficiently wide neural network with a suitable nonlinearity can approximate any continuous function on a compact set arbitrarily well. It does *not* say the network can be trained efficiently, and it does *not* say the required width, data, or parameter norms are reasonable.

Exercise 3

Exercise. Consider the toy compositional function

$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)(x_3 + x_4).$$

Identify at least one natural intermediate representation that a hierarchical model could reuse.

Solution. A natural intermediate representation is

$$u_1 = x_1 + x_2, \qquad \nu_2 = x_3 + x_4.$$

Then the output is simply $f = \nu_1\nu_2$. A hierarchical model can compute and reuse these partial sums before forming the final interaction.

Exercise 4

Exercise. Why does the proposition on ReLU networks imply that their decision boundaries can be geometrically complex even though each local piece is affine?

Solution. A ReLU network is piecewise affine: within each activation region the map is affine, but different regions have different affine pieces. The decision boundary is formed by stitching together many such pieces across regions, so globally it can be highly folded and complex even though each local patch is simple.

Exercise 5

Exercise. Compare the two viewpoints:

1. a shallow network approximates a target function in one large hidden layer;
2. a deep network approximates the same target by building intermediate representations.

What is the possible advantage of the second viewpoint?

Solution. The second viewpoint can reuse intermediate structure. Rather than approximating the whole function in one gigantic step, depth can build partial computations that are shared and recombined, which can reduce parameter count and better match compositional structure in the target.

Exercise 6

Exercise. Give one reason why approximation power alone is insufficient as a theory of practical deep learning.

Solution. Because practical deep learning depends not only on what functions exist in the class, but also on optimization, initialization, signal propagation, inductive bias, data efficiency, and generalization. Approximation theory alone says little about which good solutions are reachable or why they generalize.

11 2.4 Computation Graphs, Chain Rule, and Backpropagation

Exercise 1

Exercise. Consider the scalar computation

$$u = ax + b, \qquad v = \sigma(u), \qquad L = \frac{1}{2}(v - y)^2.$$

Compute

$$\frac{\partial L}{\partial a}, \qquad \frac{\partial L}{\partial b}$$

using the chain rule.

Solution. With $u = ax + b$, $v = \sigma(u)$, and $L = \frac{1}{2}(v - y)^2$, we have

$$\frac{\partial L}{\partial v} = v - y, \qquad \frac{\partial v}{\partial u} = \sigma'(u), \qquad \frac{\partial u}{\partial a} = x, \qquad \frac{\partial u}{\partial b} = 1.$$

Hence

$$\frac{\partial L}{\partial a} = (v - y)\sigma'(u)x, \qquad \frac{\partial L}{\partial b} = (v - y)\sigma'(u).$$

Exercise 2

Exercise. For the two-layer network

$$z^{(1)} = W_1 x + b_1, \qquad h = \sigma(z^{(1)}), \qquad \hat{y} = W_2 h + b_2,$$

derive

$$\frac{\partial L}{\partial W_2}$$

under squared loss

$$L = \frac{1}{2}\|\hat{y} - y\|^2.$$

Solution. Under squared loss,

$$L = \frac{1}{2}\|\hat{y} - y\|^2, \qquad \frac{\partial L}{\partial \hat{y}} = \hat{y} - y.$$

Since $\hat{y} = W_2 h + b_2$,

$$\frac{\partial L}{\partial W_2} = (\hat{y} - y)h^\top.$$

For a batch, this becomes the usual matrix product of output errors and hidden activations.

Exercise 3

Exercise. Using the same two-layer network, derive the hidden-layer error signal

$$\delta^{(1)} = \frac{\partial L}{\partial z^{(1)}}.$$

Explain where the Hadamard product with $\sigma'(z^{(1)})$ comes from.

Solution. Backpropagating through $\hat{y} = W_2 h + b_2$ gives

$$\frac{\partial L}{\partial h} = W_2^\top(\hat{y} - y).$$

Since $h = \sigma(z^{(1)})$, the chain rule yields

$$\delta^{(1)} = \frac{\partial L}{\partial z^{(1)}} = \left(W_2^\top(\hat{y} - y)\right) \odot \sigma'(z^{(1)}).$$

The Hadamard product appears because each coordinate of h depends only on the corresponding coordinate of $z^{(1)}$ through the pointwise nonlinearity.

Exercise 4

Exercise. Why is reverse-mode differentiation especially well suited to neural-network training, where the output of interest is a scalar loss but the number of parameters is very large?

Solution. Reverse-mode differentiation is ideal because neural-network training needs derivatives of one scalar loss with respect to many parameters. Reverse mode computes all such gradients in time on the order of a small constant multiple of the forward pass, making it vastly more efficient than differentiating separately with respect to each parameter.

Exercise 5

Exercise. Explain in words the difference between the forward pass and the backward pass in a neural network.

Solution. The forward pass computes activations from inputs to outputs and finally the loss. The backward pass propagates sensitivity information from the loss back through the computation graph so that each parameter receives its gradient.

Exercise 6

Exercise. A student says, "Backpropagation trains the network." What is incomplete or inaccurate about this statement?

Solution. Backpropagation does not itself train the network; it only computes gradients of the loss with respect to parameters. Training also requires an optimization rule such as SGD or Adam, learning-rate choices, initialization, batching, and repeated updates over data.

12 2.5 Why Deep Networks Are Hard to Train

Exercise 1

Exercise. Assume a scalar deep chain satisfies

$$\frac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}} = a \qquad \text{for every } \ell = 1, \ldots, L,$$

with constant scalar a. Show that

$$\frac{\partial L}{\partial h^{(0)}} = a^L \frac{\partial L}{\partial h^{(L)}}.$$

Analyze what happens as depth grows in the three regimes $|a| < 1$, $|a| = 1$, and $|a| > 1$.

Solution. Because each local derivative equals a,

$$\frac{\partial h^{(L)}}{\partial h^{(0)}} = \prod_{\ell=1}^{L} \frac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}} = a^L.$$

Therefore

$$\frac{\partial L}{\partial h^{(0)}} = \frac{\partial L}{\partial h^{(L)}} \frac{\partial h^{(L)}}{\partial h^{(0)}} = a^L \frac{\partial L}{\partial h^{(L)}}.$$

If $|a| < 1$, gradients vanish exponentially with depth; if $|a| > 1$, they explode; if $|a| = 1$, their scale is preserved.

Exercise 2

Exercise. Explain in words why saturation of sigmoid or tanh harms learning. Your answer should discuss both the forward-pass effect on responsiveness of activations and the backward-pass effect on gradient flow.

Solution. In the forward pass, saturated sigmoid or tanh units barely change when their input changes, so they become unresponsive and lose expressive sensitivity. In the backward pass, saturation means $\sigma'(u)$ is very small, so gradients are multiplied by tiny numbers and shrink as they pass through those units.

Exercise 3

Exercise. Compare the statements "the optimization problem is nonconvex" and "the optimization problem is ill conditioned." Why are these not the same thing? Give an example of how a nonconvex problem could still be trainable in practice, and why even a smooth problem can be slow when curvature is badly imbalanced.

Solution. Nonconvexity concerns global shape: there may be many basins and critical points. Ill conditioning concerns local geometry: level sets may be stretched so gradient descent zig-zags and progresses slowly. A nonconvex problem can still be trainable if the landscape has many good basins and optimization finds one reliably; a convex or smooth problem can still be slow if its Hessian has a very poor condition number.

13 2.6 Optimization in Practice: SGD, Initialization, and Signal Propagation

Exercise 1

Exercise. Assume a layer has m_{in} inputs, independent zero-mean weights with variance σ_W^2, and input activations of variance v. Under the same independence heuristic used in the text, compute $\text{Var}(z_i)$ for

$$z_i = \sum_{j=1}^{m_{\text{in}}} W_{ij} h_j.$$

Then determine the value of σ_W^2 that keeps $\text{Var}(z_i) \approx v$.

Solution. Under the independence heuristic,

$$z_i = \sum_{j=1}^{m_{\text{in}}} W_{ij} h_j, \qquad \mathbb{E}[z_i] = 0,$$

and cross terms vanish, so

$$\mathrm{Var}(z_i) = \sum_{j=1}^{m_{\text{in}}} \mathrm{Var}(W_{ij} h_j) = m_{\text{in}} \sigma_W^2 v.$$

To preserve variance, choose

$$m_{\text{in}} \sigma_W^2 v \approx v, \qquad \text{so} \qquad \sigma_W^2 \approx \frac{1}{m_{\text{in}}}.$$

Exercise 2

Exercise. Compare Xavier and He initialization for a network that uses tanh versus a network that uses ReLU. Why does the ReLU case call for a larger variance scale? Your answer should explicitly refer to what happens to the signal after the nonlinearity.

Solution. For tanh, activations are roughly symmetric and no large fraction is zeroed out, so Xavier scaling, about $\sigma_W^2 \sim 1/m_{\text{in}}$ (or a fan-in/fan-out compromise), keeps signal size controlled. For ReLU, about half of the mass is removed by the nonlinearity, so the post-activation second moment is roughly halved. He initialization compensates by using a larger variance, about $2/m_{\text{in}}$, to keep signal energy from shrinking layer by layer.

Exercise 3

Exercise. Reason qualitatively about minibatch size and gradient noise. Why do very small batches produce noisier updates? Why can very large batches be more stable but less exploratory? How might this tradeoff interact with learning-rate choice in practice?

Solution. Very small minibatches estimate the gradient from little data, so sampling variance is high and updates are noisy. Very large batches average away this noise, so updates are more stable and permit less stochastic exploration of the landscape. In practice, larger batches often require larger learning rates to maintain a useful update scale, while smaller batches may need smaller learning rates to avoid excessive randomness.

14 3.3 Multiscale Representation and the Fourier Viewpoint

Exercise 1

Exercise. Let $x \in \mathbb{C}^N$ and let $\delta^{(a)}$ denote the signal that is 1 at index a and 0 elsewhere. Compute $\delta^{(a)} \circledast x$ and explain why this corresponds to circular translation. How does this connect with translation equivariance?

Solution. For circular convolution with the delta at index a,

$$(\delta^{(a)} \circledast x)[n] = \sum_m \delta^{(a)}[m]x[n-m] = x[n-a] \quad \text{(indices mod } N\text{)}.$$

So the signal is shifted by a positions. This is exactly translation equivariance: translating the input translates the output.

Exercise 2

Exercise. Compute the DFT of the kernel $h = (1, -1)$ for $N = 4$ by zero-padding h to length 4. At which frequency indices does this filter have the largest magnitude response? Explain why that matches the intuition that $(1, -1)$ is a difference detector.

Solution. Zero-pad $h = (1, -1)$ to $(1, -1, 0, 0)$. Its DFT is

$$\hat{h}[k] = \sum_{n=0}^{3} h[n]e^{-2\pi ikn/4} = 1 - e^{-2\pi ik/4}.$$

Thus

$$\hat{h}[0] = 0, \quad \hat{h}[1] = 1 + i, \quad \hat{h}[2] = 2, \quad \hat{h}[3] = 1 - i.$$

The largest magnitude occurs at $k = 2$ (the highest alternating frequency), matching the intuition that $(1, -1)$ detects rapid changes rather than smooth components.

Exercise 3

Exercise. Let $h_{\text{avg}} = \frac{1}{3}(1, 1, 1)$. Show that its frequency response has largest magnitude near zero frequency. Why does this justify calling it a low-pass filter?

Solution. Its frequency response is

$$\hat{h}_{\text{avg}}(\omega) = \frac{1}{3}(1 + e^{-i\omega} + e^{-2i\omega}),$$

whose magnitude is largest at $\omega = 0$, where all terms align and sum to 1. At high frequencies the complex phases cancel more. This is why averaging is low-pass: it keeps slowly varying components and attenuates oscillatory ones.

Exercise 4

Exercise. A signal is passed through two consecutive convolutions with kernels $w^{(1)}$ and $w^{(2)}$. Use the convolution theorem to describe the combined frequency response in terms of $\widehat{w^{(1)}}$ and $\widehat{w^{(2)}}$. How does this help explain the effect of stacking convolutional layers?

Solution. If $y = x \circledast w^{(1)} \circledast w^{(2)}$, then in the Fourier domain

$$\widehat{y} = \widehat{x}\,\widehat{w^{(1)}}\,\widehat{w^{(2)}}.$$

So the combined frequency response is the product $\widehat{w^{(1)}}\widehat{w^{(2)}}$. Stacking convolutional layers therefore composes filters multiplicatively in frequency, allowing successive amplification or attenuation of different bands.

Exercise 5

Exercise. In practice many neural networks use zero padding rather than circular convolution. Explain why the discrete convolution theorem is no longer exact in the same form. Why is the Fourier viewpoint still useful anyway?

Solution. With zero padding the operator is no longer circular convolution on a finite cyclic group, so the DFT does not diagonalize it exactly in the same simple way. Still, the Fourier viewpoint remains useful because real convolutional filters are still approximately frequency-selective, and the transform still gives intuition about smoothing, edge detection, and band-pass behavior.

15 3.4 Sequential Data and Recurrent Computation

Exercise 1

Exercise. Let $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$ and $y_t = W_y h_t$. Write down the Jacobian $\partial h_t / \partial h_{t-1}$. Which two ingredients determine whether the backward signal is likely to contract or expand?

Solution. The Jacobian is

$$\frac{\partial h_t}{\partial h_{t-1}} = \mathrm{Diag}(\sigma'(W_h h_{t-1} + W_x x_t + b))W_h.$$

The two key ingredients are the recurrent weight matrix W_h and the derivative of the nonlinearity. Their norms jointly determine whether backpropagated signals contract or expand.

Exercise 2

Exercise. Suppose an RNN is used only for sequence classification, so that the loss depends only on y_T. Explain why the gradient with respect to an early hidden state h_t still depends on all later Jacobians. Relate your answer to the unrolled computation graph.

Solution. Because L depends on y_T, and y_T depends on h_T, which depends on h_{T-1}, and so on back to h_t. In the unrolled graph, the gradient from time T to time t is a product of later Jacobians, so every later recurrent transition influences how the loss reaches early states.

Exercise 3

Exercise. For the scalar recurrence $h_t = \alpha h_{t-1} + x_t$ with $h_0 = 0$, compute h_4 explicitly in terms of x_1, x_2, x_3, x_4 and α. Which inputs are weighted most strongly when $|\alpha| < 1$? Which are weighted most strongly when $|\alpha| > 1$?

Solution. Unrolling gives

$$h_1 = x_1, \quad h_2 = \alpha x_1 + x_2, \quad h_3 = \alpha^2 x_1 + \alpha x_2 + x_3,$$

and hence

$$h_4 = \alpha^3 x_1 + \alpha^2 x_2 + \alpha x_3 + x_4.$$

If $|\alpha| < 1$, recent inputs are weighted more strongly; if $|\alpha| > 1$, older inputs are amplified more strongly (often unstably).

Exercise 4

Exercise. Assume that $\|A_k\| \leq \rho < 1$ for all k. Show that the contribution of the loss at time s to the gradient at time t is bounded by a constant times ρ^{s-t}. Interpret this bound in words.

Solution. The gradient contribution from time s to time t contains a product $A_s A_{s-1} \cdots A_{t+1}$. Therefore

$$\|A_s \cdots A_{t+1}\| \leq \prod_{k=t+1}^{s} \|A_k\| \leq \rho^{s-t}.$$

Up to a constant from the terminal derivative, the influence is bounded by $C\rho^{s-t}$. In words, when each local Jacobian contracts, influence decays exponentially with temporal distance.

Exercise 5

Exercise. Why is it reasonable to describe an RNN as depth with weight sharing across time? In what sense is this analogous to, and different from, weight sharing in a convolutional network?

Solution. An RNN is "depth through time" because repeated application of the same transition map over T steps yields an unrolled depth-T computation. This is analogous to CNN weight sharing in that the same parameters are reused repeatedly, but different because CNN sharing is across spatial locations within one layer, whereas RNN sharing is across time steps in a sequential state evolution.

16 3.5 Gating and the Control of Memory

Exercise 1

Exercise. Write the LSTM cell-state update in coordinate form. For a fixed coordinate j, explain in words what happens when $(f_t)_j$ is close to 1 and $(i_t)_j$ is close to 0. What happens when $(f_t)_j$ is close to 0 and $(i_t)_j$ is large?

Solution. Coordinatewise, an LSTM cell update is

$$(c_t)_j = (f_t)_j (c_{t-1})_j + (i_t)_j (\tilde{c}_t)_j.$$

If $(f_t)_j \approx 1$ and $(i_t)_j \approx 0$, the old memory is nearly preserved. If $(f_t)_j \approx 0$ and $(i_t)_j$ is large, the old content is mostly erased and replaced by new candidate information.

Exercise 2

Exercise. Consider the recurrence $m_t = g_t m_{t-1} + u_t$ in one dimension. Show that

$$m_t = \Big(\prod_{k=1}^{t} g_k\Big) m_0 + \sum_{s=1}^{t} \Big(\prod_{k=s+1}^{t} g_k\Big) u_s.$$

Interpret this formula as a weighted memory of past updates.

Solution. Prove by induction. For $t = 1$, $m_1 = g_1 m_0 + u_1$, which matches the formula. Assuming it holds at $t-1$,

$$m_t = g_t \Big(\prod_{k=1}^{t-1} g_k\Big) m_0 + g_t \sum_{s=1}^{t-1} \Big(\prod_{k=s+1}^{t-1} g_k\Big) u_s + u_t,$$

which simplifies to the stated expression. Interpretation: each past update u_s is remembered with weight $\prod_{k=s+1}^{t} g_k$, so later gates decide how much of each past input survives.

Exercise 3

Exercise. In a GRU, the update equation is $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \widetilde{h}_t$. Describe the behavior of the model when z_t is close to 0 coordinatewise. Describe the behavior when z_t is close to 1. How does this compare to the role of the forget and input gates in an LSTM?

Solution. If $z_t \approx 0$, then $h_t \approx h_{t-1}$ and the GRU mostly keeps its previous state. If $z_t \approx 1$, then $h_t \approx \tilde{h}_t$ and the state is mostly overwritten by the candidate. This blends preservation and writing in one gate, whereas the LSTM separates these roles through distinct forget and input gates.

Exercise 4

Exercise. Suppose a one-dimensional LSTM-like memory satisfies

$$c_t = 0.99c_{t-1}$$

for $t = 2, \ldots, 101$ after writing a signal at time 1. Compute $\partial c_{101}/\partial c_1$. Repeat the same calculation for a plain scalar recurrence whose local derivative is constantly 0.8. Compare the two values.

Solution. For the LSTM-like memory,

$$\frac{\partial c_{101}}{\partial c_1} = 0.99^{100} \approx 0.366.$$

For the plain recurrence with local derivative 0.8,

$$\frac{\partial h_{101}}{\partial h_1} = 0.8^{100} \approx 2.0 \times 10^{-10}.$$

So the gated memory preserves gradient vastly better.

Exercise 5

Exercise. From the viewpoint of discrete-time dynamical systems, explain one important difference between a plain RNN and an LSTM, and one important difference between an LSTM and a GRU. Your answer should refer explicitly to the structure of the state update rather than to historical popularity or benchmark performance.

Solution. A plain RNN updates all state coordinates through one recurrent nonlinearity, so memory and transformation are entangled. An LSTM introduces an explicit cell state with gated additive updates, allowing near-identity transport of memory. A GRU is simpler than an LSTM: it still gates updates, but it folds some of the memory-control roles into fewer state variables and gates.

17 3.6 From Recurrence to Attention

Exercise 1

Exercise. Let values $v_1, v_2, v_3 \in \mathbb{R}^d$ and weights $\alpha_1, \alpha_2, \alpha_3$ satisfy $\alpha_i \geq 0$ and $\alpha_1 + \alpha_2 + \alpha_3 = 1$. Show directly that $\sum_i \alpha_i v_i$ lies in the convex hull of $\{v_1, v_2, v_3\}$. Why does this apply automatically to softmax attention?

Solution. Because $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$, the point

$$\alpha_1 v_1 + \alpha_2 v_2 + \alpha_3 v_3$$

is by definition a convex combination of v_1, v_2, v_3, hence lies in their convex hull. In softmax attention the row weights are always nonnegative and sum to one, so each context vector is automatically a convex combination of value vectors.

Exercise 2

Exercise. Suppose a decoder step produces similarity scores $(1, 2, 0)$ against three source positions. Compute the corresponding softmax weights. Which source position receives the largest share of attention?

Solution. The softmax of $(1, 2, 0)$ is

$$\Big(\frac{e^1}{e^1+e^2+e^0}, \frac{e^2}{e^1+e^2+e^0}, \frac{e^0}{e^1+e^2+e^0}\Big) \approx (0.245, 0.665, 0.090).$$

So the second source position receives the largest share of attention.

Exercise 3

Exercise. Explain in your own words why the vector h_T in a classical encoder–decoder model is a bottleneck. Your answer should refer both to representation capacity and to long-range dependence through recurrent updates.

Solution. h_T is a bottleneck because it must compress the entire source sequence into one fixed-size vector. This limits representation capacity for long or information-rich inputs, and in an RNN it also requires information from early tokens to survive many recurrent updates before reaching the final state.

Exercise 4

Exercise. In a toy translation example with source words (x_1, x_2, x_3) and target words (y_1, y_2), suppose the alignment weights are

$$(0.7, 0.2, 0.1) \quad \text{for } y_1, \qquad (0.1, 0.1, 0.8) \quad \text{for } y_2.$$

Describe qualitatively what this says about the different context vectors used at the two decoder steps.

Solution. For y_1, the weights $(0.7, 0.2, 0.1)$ mean the first source word dominates the context, with smaller contributions from the others. For y_2, $(0.1, 0.1, 0.8)$ means the decoder now focuses mostly on the third source word. So the two decoder steps consult different source-side information rather than using one fixed summary.

Exercise 5

Exercise. Why is the weighted-sum form of attention more compatible with gradient-based training than a hard rule that selects exactly one memory slot by an argmax operation? Give both an optimization-based and a representational answer.

Solution. Optimization: a weighted sum with softmax is differentiable, so gradients flow through all memory locations and the attention scores. An argmax is discontinuous and blocks ordinary gradient flow. Representation: soft attention can blend multiple relevant memories, whereas hard selection forces a single slot and loses partial or distributed context.

18 3.7 Transformers as General-Purpose Sequence Architectures

Exercise 1

Exercise. Let $X \in \mathbb{R}^{T\times d}$ and let P be a permutation matrix. Write out carefully why $QK^\top$ transforms as $P(QK^\top)P^\top$ under the substitution $X \mapsto PX$.

Solution. If $X' = PX$, then $Q' = PQ$ and $K' = PK$. Hence

$$Q'K'^\top = (PQ)(PK)^\top = PQK^\top P^\top.$$

Thus the score matrix is conjugated by the permutation. Row-wise softmax preserves this conjugation, giving the permutation-equivariance statement.

Exercise 2

Exercise. Suppose all token representations in a self-attention layer are identical and no positional information is added. Show that every row of the attention matrix is the same. What does this imply about the outputs?

Solution. If all token representations are identical, then all queries are equal and all keys are equal. Therefore every row of $QK^\top$ is identical, and so every row of the attention matrix is identical after row-wise softmax. Since the values are also identical across positions up to the same transformation, all outputs are the same vector at every position.

Exercise 3

Exercise. For the toy example in this section, replace

$$Q = K = [1, 0, 1]^\top$$

by

$$Q = K = [2, 0, -2]^\top.$$

Compute the new attention matrix and explain how the sharper score differences change the interaction pattern.

Solution. With scalar keys/queries and no scaling for simplicity, the score matrix is

$$S = QK^\top = \begin{bmatrix} 4 & 0 & -4 \\ 0 & 0 & 0 \\ -4 & 0 & 4 \end{bmatrix}.$$

Applying row-wise softmax gives approximately

$$A \approx \begin{bmatrix} 0.982 & 0.018 & 0.0003 \\ 1/3 & 1/3 & 1/3 \\ 0.0003 & 0.018 & 0.982 \end{bmatrix}.$$

Compared with $[1, 0, 1]^\top$, the larger score magnitudes make the first and third rows much sharper: each strongly attends to itself/opposite-sign relation and almost ignores the mismatched token.

Exercise 4

Exercise. Compare a recurrent layer and a self-attention layer as mechanisms for modeling a dependency between the first and last token of a long sequence. Which architecture has the shorter interaction path, and which has the cheaper asymptotic dependence on sequence length?

Solution. Self-attention has interaction path length 1 between the first and last token in one layer, because token T can attend directly to token 1. A recurrent layer requires information to traverse $T - 1$ state transitions, so the path length grows with sequence length. Asymptotically, recurrence is cheaper per layer in T (roughly linear), while dense self-attention is more expensive (quadratic) because it compares all token pairs.

Exercise 5

Exercise. Derive the output dimensions of a multi-head attention block with H heads, model width d, per-head key dimension d_k, and per-head value dimension d_v.

Solution. Each head produces an output of shape $T \times d_v$. Concatenating H heads gives a matrix of shape $T \times (Hd_v)$. The output projection therefore has shape $(Hd_v) \times d$ if the final model width is d, and the final block output has shape $T \times d$.

19 3.8 Scaling Transformer Architectures and Multimodal Extensions

Exercise 1

Exercise. Starting from the cost of one dense self-attention layer, derive the leading-order complexity $O(LTd^2 + LT^2d)$ for a stack of L layers. State clearly which terms come from projections, attention scores, and value aggregation.

Solution. For one layer, forming Q, K, V costs $O(Td^2)$ (three projections of width d), computing scores $QK^\top$ costs $O(T^2d)$, and multiplying attention weights by V also costs $O(T^2d)$ up to constants. So one dense layer is $O(Td^2 + T^2d)$, and a stack of L layers is

$$O(LTd^2 + LT^2d).$$

The first term comes from projections and feedforward-like linear maps; the second from all-pairs token interactions.

Exercise 2

Exercise. Compare an encoder-only architecture and a decoder-only architecture on the same input sequence of length T. Which one permits token i to depend on token $j > i$ in a single layer? Explain the modeling consequence.

Solution. In an encoder-only architecture, token i can depend on token $j > i$ in a single layer because attention is typically bidirectional. In a decoder-only architecture with causal masking, token i cannot depend on future token $j > i$ in that layer. Consequently, encoder-only models are natural for full-context understanding, while decoder-only models are natural for autoregressive generation.

Exercise 3

Exercise. Let $Y \in \mathbb{R}^{T_y \times d}$ and $Z \in \mathbb{R}^{T_x \times d}$. Derive the dimensions of the matrices Q, K, V, the cross-attention score matrix, and the final cross-attention output.

Solution. For cross-attention from target-side queries Y to source-side keys/values Z:

$$Q = YW_Q \in \mathbb{R}^{T_y \times d_k}, \quad K = ZW_K \in \mathbb{R}^{T_x \times d_k}, \quad V = ZW_V \in \mathbb{R}^{T_x \times d_v}.$$

So the score matrix $QK^\top$ has shape $T_y \times T_x$, and the final output $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$ has shape $T_y \times d_v$.

Exercise 4

Exercise. Consider a multimodal system with text tokens $Z^{(t)} \in \mathbb{R}^{T_t \times d}$ and image tokens $Z^{(i)} \in \mathbb{R}^{T_i \times d}$. Compare early fusion by concatenation with fusion by cross-attention. What interaction patterns are possible in each design?

Solution. Early fusion by concatenation creates one long token sequence of length $T_t + T_i$ and allows any later attention layer to form unrestricted interactions among all tokens. Cross-attention keeps the modalities partly distinct: one stream queries another, so interaction is directed and structured rather than fully symmetric from the start.

Exercise 5

Exercise. Suppose the context length is doubled while depth and width are fixed. How do the leading projection term and the leading dense-attention interaction term change asymptotically?

Solution. If T doubles to $2T$, the projection term $O(Td^2)$ doubles, while the dense interaction term $O(T^2d)$ grows by a factor of four. This is why long-context scaling is dominated by attention interactions rather than linear projections.

Exercise 6

Exercise. Give one advantage and one disadvantage of local-window attention compared with dense attention. Your answer should distinguish computational benefit from possible representational loss.

Solution. Advantage: local-window attention reduces computation and memory, often from quadratic to near-linear in sequence length for fixed window size. Disadvantage: it may miss long-range dependencies unless additional mechanisms are added, so representational reach is reduced compared with dense attention.

20 3.9 Worked Examples and Comparative Case Studies

Exercise 1

Exercise. In the CNN example, replace the filter

$$\begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

by

$$\begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}.$$

Compute the new feature map and explain what pattern this second filter is designed to detect.

Solution. With the toy image from the section, the new filter

$$\begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

compares the top row of each patch to the bottom row, so it detects *horizontal* edges. Because the example image is constant across rows, every 2×2 patch has equal top and bottom sums, and the resulting feature map is all zeros:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Exercise 2

Exercise. For the toy memory task, explain from first principles why the information path from the first token to the last output has length on the order of T in a recurrent model but length 1 in a self-attention layer.

Solution. In an RNN, information from the first token must be carried through the chain of intermediate hidden states all the way to the last step, so the path length is proportional to T. In self-attention, the last token can directly attend to the first token in one interaction, so the path length is 1.

Exercise 3

Exercise. In the translation example, verify numerically that each row of the alignment matrix A sums to one. Then explain why this implies that each context vector is a convex combination of value vectors.

Solution. Each row of an alignment matrix produced by row-wise softmax sums to one by construction. Therefore each context vector

$$c_i = \sum_j A_{ij} v_j$$

uses nonnegative weights summing to one, so c_i is a convex combination of the value vectors. This is exactly the geometric interpretation of soft attention.

Exercise 4

Exercise. Compare the recurrent forecasting rule

$$h_t = 0.7h_{t-1} + 0.3x_t$$

with the causal convolutional rule

$$\hat{x}_{t+1} = 0.5x_t + 0.5x_{t-1}.$$

What hidden assumption about temporal structure is built into each method?

Solution. The recurrent rule assumes temporal state accumulation: the hidden state is a running summary of the past, with exponentially decaying memory. The causal convolutional rule assumes that the next value depends on a fixed local window of recent observations with fixed receptive field and no adaptive hidden memory.

Exercise 5

Exercise. Give one task for which a CNN is likely to be more natural than a transformer, and one task for which a transformer is likely to be more natural than a plain RNN. Justify both answers in terms of inductive bias rather than popularity.

Solution. A CNN is more natural for image edge or texture detection because locality and translation sharing match the spatial structure. A transformer is more natural than a plain RNN for long-context language modeling, because it can form direct content-dependent interactions across distant tokens rather than compressing everything through a single recurrent state.

Exercise 6

Exercise. Choose any two rows from Table **??** and write a short proof-style argument explaining why their operators impose genuinely different constraints on how information can move through the network.

Solution. For example, compare convolution and self-attention. Convolution restricts token interaction to local neighborhoods and shares the same filter across positions, so information movement is local and translation-structured. Dense self-attention permits any position to interact with any other in one layer via content-based weights, so its operator imposes a much richer but less strongly biased interaction pattern. This difference is structural, not merely implementational.

21 3.10 What Is Understood and What Remains Open

Exercise 1

Exercise. Pick three entries from Table **??** and classify each one as a theorem, a proposition-level structural statement, an exact architectural identity, or an asymptotic derivation. Explain why these are different kinds of mathematical knowledge.

Solution. One possible classification is: the convolution theorem is a *theorem*; permutation equivariance of self-attention without positional encoding is a *proposition-level structural statement*; the matrix form of a transformer block is an *exact architectural identity*; and the complexity formula $O(LTd^2 + LT^2d)$ is an *asymptotic derivation*. These differ because some are exact mathematical results, some characterize architecture structure, and some describe scaling laws rather than equality statements.

Exercise 2

Exercise. Compare convolution and self-attention as inductive biases. In what sense does convolution build in stronger prior assumptions than dense self-attention? In what sense does self-attention allow a richer interaction family?

Solution. Convolution builds in stronger prior assumptions because it enforces locality and translation equivariance by design. Dense self-attention assumes much less about spatial or temporal locality and can represent a wider range of pairwise interactions. So convolution is more biased but often more data-efficient when that bias is correct, while self-attention is more flexible.

Exercise 3

Exercise. Starting from the recurrence $h_t = \phi(W_h h_{t-1} + W_x x_t)$, explain why the path length from token 1 to token T grows with T. Then compare this with the direct interaction path in self-attention.

Solution. In the recurrence $h_t = \phi(W_h h_{t-1} + W_x x_t)$, information from token 1 reaches token T only through repeated composition across $T - 1$ state transitions, so the path length grows with T. In self-attention, token T can connect directly to token 1 within one layer through an attention edge, so the path length is constant per layer.

Exercise 4

Exercise. Give a concrete example of a domain in which explicit symmetry or equivariance should guide architecture design. Explain what would likely be lost if one used a completely generic token model without encoding that structure.

Solution. A good example is molecular modeling or 3D point-cloud processing, where rotation, translation, or permutation symmetries matter. If one ignores this structure and uses a completely generic token model, the architecture may waste data learning symmetries that could have been built in, and predictions may be less stable under physically irrelevant transformations.

Exercise 5

Exercise. "Transformers succeeded, so architecture no longer matters." Critically evaluate this statement using ideas from multimodal interfaces, long-context modeling, and geometric deep learning.

Solution. The statement is false. Transformers are powerful, but architecture still matters because modality, geometry, and computational budget differ. Long-context modeling often needs sparse or structured attention; multimodal systems must decide how to fuse modalities; geometric data often benefit from equivariant operators. Success of one family does not erase the value of matched inductive bias.

Exercise 6

Exercise. Choose one category from the annotated bibliography and trace a short historical arc from a foundational paper to a modern architectural family. Identify what stayed conceptually stable and what changed in implementation practice.

Solution. For example, one can trace attention from Bahdanau-style alignment in encoder–decoder translation to the transformer family. Conceptually stable ideas include content-based weighted aggregation and direct access to distributed memory. What changed in practice is scale: multi-head blocks, residual stacks, normalization, pretraining regimes, and hardware-aware implementations turned the basic idea into a general-purpose architecture.

22 4.1 From Task-Specific Prediction to Reusable Representation

Exercise 1

Exercise. Let $\phi : X \to Z$ be fixed and let $\mathcal{G}$ be a downstream hypothesis class on Z. Prove carefully that empirical risk minimization over $\mathcal{G}$ on transformed data is equivalent to empirical risk minimization over $\mathcal{H}_\phi = \{g \circ \phi : g \in \mathcal{G}\}$ on the original data.

Solution. On transformed data, ERM chooses

$$\hat{g} \in \arg\min_{g \in \mathcal{G}} \frac{1}{n} \sum_{i=1}^{n} \ell(g(\phi(x_i)), y_i).$$

But every $g \in \mathcal{G}$ corresponds to $h = g \circ \phi \in \mathcal{H}_\phi$, and conversely every $h \in \mathcal{H}_\phi$ has that form. Therefore

$$\min_{g \in \mathcal{G}} \frac{1}{n} \sum_{i} \ell(g(\phi(x_i)), y_i) = \min_{h \in \mathcal{H}_\phi} \frac{1}{n} \sum_{i} \ell(h(x_i), y_i),$$

so the optimization problems are identical after the change of variables $h = g \circ \phi$.

Exercise 2

Exercise. Give an example of a feature that is useful for one classification task but unlikely to transfer well to a broader family of tasks. Explain why it is task-specific rather than reusable.

Solution. A feature such as "contains the watermark of hospital A" might help classify images from one hospital's workflow but transfer poorly to broader medical tasks. It is task-specific because it exploits a narrow dataset artifact rather than representing reusable semantic structure like anatomy or lesion shape.

Exercise 3

Exercise. In the concentric-circle example, verify that no affine classifier on $\mathbb{R}^2$ can separate the inner circle from the outer circle, but that a threshold on $\|x\|^2$ can.

Solution. No affine classifier can separate concentric circles because any affine decision boundary in $\mathbb{R}^2$ is a line, which cannot isolate a bounded inner ring from an outer ring surrounding it. But the feature $r^2 = \|x\|^2$ separates them immediately: choose a threshold τ between the squared inner and outer radii and classify by whether $\|x\|^2 < \tau$.

Exercise 4

Exercise. Suppose $\phi : \mathbb{R}^d \to \mathbb{R}^m$ is learned and a downstream task uses a linear probe. Write the resulting predictor explicitly and explain why it may still define a nonlinear decision boundary in the original input space.

Solution. A linear probe on the learned representation has form

$$f(x) = w^\top \phi(x) + b.$$

This is linear in feature space, but unless ϕ itself is affine, the map $x \mapsto w^\top \phi(x) + b$ is generally nonlinear in the original input space, so the induced decision boundary in input coordinates can be curved or highly nontrivial.

Exercise 5

Exercise. Compare two adaptation strategies after pretraining: freezing ϕ and training only g, versus fine-tuning both ϕ and g. What are the likely advantages and risks of each?

Solution. Freezing ϕ and training only g is cheaper, reduces overfitting risk, and preserves general reusable features, but it may underadapt to the new task. Fine-tuning both ϕ and g can yield better task performance by reshaping the representation, but it costs more data/compute and risks forgetting or overfitting.

23 4.5 Representation Learning as a Unifying Principle

Exercise 1

Exercise. Give an example where increasing invariance reduces performance due to loss of sufficiency.

Solution. Face recognition is a good example: making the representation invariant to identity-preserving pose changes is useful, but making it invariant to *person identity* would destroy the task. More generally, pushing invariance too far can remove task-relevant information and hence violate sufficiency.

Exercise 2

Exercise. Show that if ϕ is invertible, it is always sufficient but not necessarily invariant.

Solution. If ϕ is invertible, no information is lost: from $\phi(x)$ one can recover x, so any target that was predictable from x is equally predictable from $\phi(x)$; hence ϕ is sufficient. But invertibility does not imply invariance to a nuisance transformation g, because typically $\phi(gx) \neq \phi(x)$ unless g acts trivially.

Exercise 3

Exercise. Compare representations learned by supervised and contrastive objectives on the same dataset.

Solution. Supervised representations are shaped to preserve distinctions relevant to the labeled task and may discard information unrelated to that label. Contrastive representations are shaped to make different views of the same instance close while separating different instances, often producing more generic geometry useful across tasks. So supervised learning is often more label-specific, while contrastive learning can encourage broader transfer.

Exercise 4

Exercise. Construct a simple RL example where a poor representation violates the Markov property.

Solution. Suppose the true environment state is (p, v), position and velocity, but the learned representation keeps only position p. At the same observed p, moving left and moving right lead to different next positions, so $P(s_{t+1} \mid \phi(o_t), a_t)$ is not well defined from the reduced representation. The Markov property fails because hidden velocity matters.

24 4.6 State Representation in Sequential Decision-Making

Exercise 1

Exercise. Compute the value function for a three-state chain with deterministic transitions.

Solution. A simple three-state deterministic chain is $s_1 \to s_2 \to s_3 \to s_3$ with rewards r_1, r_2, r_3 and discount γ. Then

$$V(s_3) = r_3 + \gamma V(s_3) = \frac{r_3}{1-\gamma},$$

$$V(s_2) = r_2 + \gamma V(s_3), \qquad V(s_1) = r_1 + \gamma V(s_2).$$

Substituting gives the full value function. For example, if $r_1 = 1$ and $r_2 = r_3 = 0$, then $V(s_3) = 0$, $V(s_2) = 0$, and $V(s_1) = 1$.

Exercise 2

Exercise. Show that policy evaluation defines a linear system in V^π.

Solution. For a fixed policy π, the Bellman expectation equation is linear in V^π:

$$V^\pi = r_\pi + \gamma P_\pi V^\pi,$$

where $r_\pi(s) = \sum_a \pi(a \mid s) R(s,a)$ and $P_\pi(s,s') = \sum_a \pi(a \mid s) P(s' \mid s,a)$. Rearranging gives

$$(I - \gamma P_\pi) V^\pi = r_\pi,$$

which is a linear system.

Exercise 3

Exercise. Prove that the Bellman expectation operator is also a contraction.

Solution. Let $(\mathcal{T}_\pi V)(s) = \sum_a \pi(a \mid s)\big[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s')\big]$. Then

$$|(\mathcal{T}_\pi V)(s) - (\mathcal{T}_\pi W)(s)| \le \gamma \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s,a) |V(s') - W(s')| \le \gamma \|V - W\|_\infty.$$

Taking the supremum over s yields

$$\|\mathcal{T}_\pi V - \mathcal{T}_\pi W\|_\infty \le \gamma \|V - W\|_\infty.$$

Exercise 4

Exercise. Construct an example where a poor representation violates the Markov property.

Solution. An observation may show only a cropped view of a moving object, while the hidden full state also includes its velocity. If the learned representation keeps only the crop, then the same representation can lead to different next observations depending on the unseen velocity. Thus the reduced state is not Markov.

25 4.7 Deep Reinforcement Learning as Learned Representation Plus Control

Exercise 1

Exercise. Derive the TD update for V_θ using the Bellman expectation equation.

Solution. Starting from the Bellman expectation equation, a one-step bootstrap target is

$$y_t = r_t + \gamma V_\theta(s_{t+1}).$$

Using squared TD error gives loss $\frac{1}{2}(V_\theta(s_t) - y_t)^2$, so a semi-gradient update is

$$\theta \leftarrow \theta + \alpha\big(r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)\big)\nabla_\theta V_\theta(s_t).$$

Exercise 2

Exercise. Show how the policy gradient identity changes when using a baseline.

Solution. With a baseline $b(s_t)$, the policy-gradient identity becomes

$$\nabla J(\theta) = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\big(G_t - b(s_t)\big)\Big].$$

The baseline does not change the expectation as long as it does not depend on a_t; it reduces variance by centering the return.

Exercise 3

Exercise. Construct an example where model-based RL fails due to model bias.

Solution. Suppose a robot model predicts frictionless motion, but the true environment has friction and occasional slipping. Planning in the learned model recommends aggressive long moves that look high reward in simulation but fail in reality. This is model bias: errors in the learned dynamics systematically mislead planning.

Exercise 4

Exercise. Explain how representation learning differs between supervised learning and RL.

Solution. In supervised learning the target signal is usually given directly by labels, so representation learning is organized around immediate predictive discrimination. In RL the representation must support control, delayed reward, credit assignment, and often approximate Markov state construction. So RL representations are shaped by decision relevance and future consequences, not just label prediction.

26 5.4 Adversarial Generative Modeling

Exercise 1

Exercise. Derive the optimal discriminator formula by maximizing the integrand pointwise for fixed p_{data} and p_G.

Solution. For fixed p_{data} and p_G, maximize pointwise the function

$$\phi_x(D) = p_{\text{data}}(x) \log D + p_G(x) \log(1 - D), \qquad 0 < D < 1.$$

Differentiating and setting to zero,

$$\frac{p_{\text{data}}(x)}{D} - \frac{p_G(x)}{1 - D} = 0 \quad \Longrightarrow \quad D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}.$$

The second derivative is negative, so this is indeed the maximizer.

Exercise 2

Exercise. Show that if $p_G = p_{\text{data}}$, then $D^*(x) = \frac{1}{2}$ and the optimal value is $-\log 4$.

Solution. If $p_G = p_{\text{data}}$, then

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{\text{data}}(x)} = \frac{1}{2}.$$

Plugging into the value function gives

$$V(D^*, G) = \int p_{\text{data}}(x) \log \tfrac{1}{2}\, dx + \int p_G(x) \log \tfrac{1}{2}\, dx = -\log 2 - \log 2 = -\log 4.$$

Exercise 3

Exercise. Why can the original minimax generator loss produce weak gradients when the discriminator is very strong? Explain without computation and then verify by differentiating the loss formally.

Solution. If the discriminator is very strong, fake samples get $D(G(z)) \approx 0$. Under the original minimax generator objective, the generator minimizes $\log(1 - D(G(z)))$, whose slope with respect to generator parameters becomes weak when the discriminator is near saturation. Formally,

$$\nabla_\theta \log(1 - D(G_\theta(z))) = -\frac{1}{1 - D} \nabla_\theta D(G_\theta(z)),$$

and when D saturates near 0, the internal sigmoid derivatives in $\nabla_\theta D$ can become tiny, producing weak learning signals. This motivates the non-saturating alternative.

Exercise 4

Exercise. Compare a VAE and a GAN in terms of: (i) tractable likelihood, (ii) sampling path, (iii) training stability, and (iv) latent-space interpretation.

Solution. (i) VAE: tractable variational lower bound on likelihood; GAN: no direct tractable likelihood in the basic form. (ii) VAE sampling uses latent variable sampling followed by a decoder; GAN sampling pushes noise through a generator. (iii) VAEs are often more stable but can blur samples; GANs often produce sharper samples but are harder to train. (iv) VAE latents are usually more explicitly structured and inference-friendly; GAN latents can be meaningful but are not tied to a tractable encoder-likelihood framework.

Exercise 5

Exercise. In a conditional GAN, explain carefully what distribution is being matched when the condition variable c is fixed.

Solution. When the condition c is fixed, a conditional GAN aims to match the conditional data distribution $p_{\text{data}}(x \mid c)$ by the generator distribution $p_G(x \mid c)$. Equivalently, across varying c it aims to match the joint distribution induced by $p(c)p_G(x \mid c)$ to the data joint $p(c)p_{\text{data}}(x \mid c)$, assuming the same condition distribution is used.